PROJECT REPORT

Lumina CV

Noise- and Temporal-Aware
Zero-Reference Low-Light Image Enhancement

Jatin Purbia	B23CH11022
Sharana Gouda Lakshetty	B23CH1042
Rahul Ahuja	B23CH1037

November 2026

Abstract

We built Lumina CV, an end-to-end system for brightening photographs taken in poor light. It has three parts: a PyTorch model ("NTA-ZeroDCE"), a FastAPI inference server and a Next.js web application that shows a before/after comparison with an in-browser quality analysis. The model extends Zero-Reference Deep Curve Estimation (Zero-DCE) with a noise-estimation branch, a residual denoiser, and extra losses for edges, chromaticity and adaptive exposure. It also has an optional video-consistency path that uses optical flow. It is trained without any paired ground truth.

This report describes the method and the software, and it analyses the training settings we tried while looking for the best model. We evaluated all eight saved training checkpoints (epochs 5 to 40) and four inference settings on the 15 held-out LOL image pairs. We also compared the model against plain gamma correction and histogram equalisation. The main finding is a trade-off. Held-out PSNR rises from 10.7 dB (epoch 10) to 16.6 dB (epoch 40), but the model drifts towards brightening everything too much, so ordinary photographs wash out. We therefore ship the 10-epoch model (exposure target 0.6). Gamma correction with $\gamma = 0.3$ reaches 16.7 dB PSNR, higher than any checkpoint, so the model's advantage is in colour fidelity and noise behaviour rather than pixel-wise PSNR.

We followed up with two concrete changes. First, we built a reusable held-out-photo validation pipeline (a 48-scene normal-light preservation set carved out of the fixed training split, plus a per-image regression gate on brightness, contrast and clipping) so that a brightening regression can be *measured* before a checkpoint ships, not only noticed by eye. Second, we used it to test the obvious mitigation: retraining for 10 epochs with the exposure target lowered from 0.6 to 0.5. The candidate passed the preservation gate and improved normal-photo structural similarity slightly, but it also *lowered* held-out low-light PSNR (10.68→9.51 dB) and SSIM (0.506→0.415) and visibly lost shadow detail, so we did not promote it; the shipped model keeps the original 0.6 target. We also found and fixed a real reliability bug: the deployed Starlette version silently broke the API's

multipart upload parser, so every enhancement request failed; pinning a newer Starlette fixed it, and the regression test suite (now 2,080 passing, 2 skipped) covers it. The 13-row ablation matrix remains implemented but not run, so we still make no ablation claims.

Contents

1 Introduction **4**

2 Method **4**

2.1 Notation and curve model 4

2.2 Network architecture 4

2.3 Training losses 5

2.4 Video consistency 6

3 System implementation **6**

4 Experimental setup **7**

5 Results **8**

5.1 Training dynamics 8

5.2 Tuning the training length (checkpoint sweep) 9

5.3 Why we nevertheless ship the 10-epoch model 9

5.4 A reusable preservation gate for checkpoint promotion 10

5.5 Testing the exposure-target mitigation 11

5.6 Fixing a real API reliability bug 11

5.7 Comparison with simple baselines 12

5.8 What the model learns: curves, noise map and denoiser 12

5.9 Inference-time setting: `curve_scale` 13

5.10 Other settings we tried 14

6 Discussion **15**

7 Conclusion **16**

A Hyperparameters of the shipped configuration **18**

1 Introduction

Photographs taken in low light are dark, noisy and low in contrast. Brightening them is simple to state but hard to do well: a global brightness boost lifts the noise as much as the signal, and it can blow out regions that were already well exposed. Learning-based enhancers address this but usually need paired dark/bright images, which are expensive to collect and tie the model to one camera.

Zero-DCE [1] avoids paired data. A small network predicts, for every pixel, the parameters of a brightness curve. The curve is applied several times to the input. Training uses only hand-designed losses (exposure, colour constancy, spatial consistency, smoothness) computed on the output itself. The network is tiny (about 1 MB here) and fast.

Our goals were:

1. Re-implement the Zero-DCE idea cleanly, with a strict configuration system and a large automated test-suite.
2. Add *noise awareness*, so that dark regions are not brightened into visible noise, and a light denoiser.
3. Add edge, chromaticity and adaptive-exposure losses, and an optional temporal-consistency mechanism for video.
4. Wrap the model in an HTTP API and a web application anyone can use, with an in-browser analysis of what the enhancement did.
5. Find a training recipe that gives natural results, and document honestly what worked and what did not.

Contributions. (i) The NTA-ZeroDCE model and training code; (ii) a deployable API and web app; (iii) an evaluation of the training schedule, baselines and inference settings, including the finding that longer training over-brightens ordinary photos while improving the standard benchmark score.

2 Method

2.1 Notation and curve model

Images are RGB tensors with values in $[0, 1]$. The network $\mathcal{F}$ predicts eight curve-parameter maps $A_1, \ldots, A_8$ (each with three channels, so 24 output channels, passed through tanh to lie in $[-1, 1]$). The input $x_0 = x$ is brightened by iterating the quadratic curve

$$x_{i+1} = x_i + A_i \odot x_i \odot (1 - x_i), \qquad i = 0, \ldots, 7. \tag{1}$$

The curve fixes the end points 0 and 1 for any A, and $A = 0$ is the identity (Fig. 1). Positive A brightens, negative A darkens, and because A varies per pixel the enhancement is spatially adaptive.

2.2 Network architecture

Fig. 2 shows the pipeline. All components are small convolutional networks.

- **DCE-Net.** Seven 3×3 convolutions with 32 channels, ReLU, symmetric skip connections ($h_3{+}h_4$, $h_2{+}h_5$, $h_1{+}h_6$) and a tanh head. No normalisation or down-sampling.
- **Noise branch.** Three convolutions (RGB $\to 16 \to 16 \to 1$) with a sigmoid scaled to $[0, 0.2]$; it predicts a per-pixel noise level σ, which is concatenated to the DCE-Net input.
- **Residual denoiser.** Two depthwise/pointwise 16-channel blocks. Its input is the curve output z plus $\sigma \cdot G$, where $G = \mathrm{clamp}(\mathrm{avgpool}_5(Y(z))/\mathrm{avgpool}_5(Y(x)), 1, 8)$ is the local gain

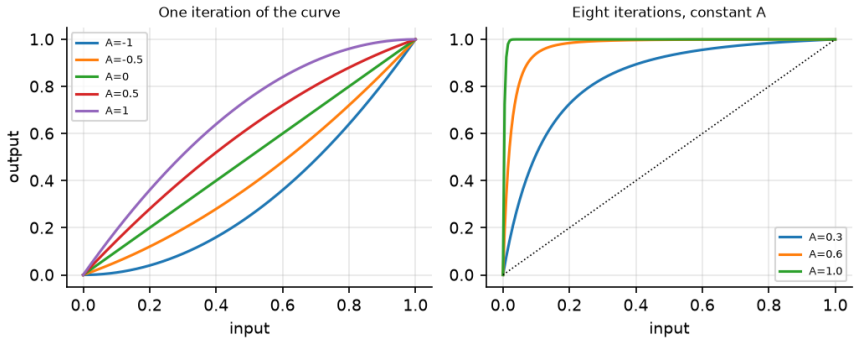


Figure 1: The light-enhancement curve. Left: one iteration for several values of A. Right: eight iterations with a constant A give a strong, smooth tone-mapping that never leaves $[0, 1]$.

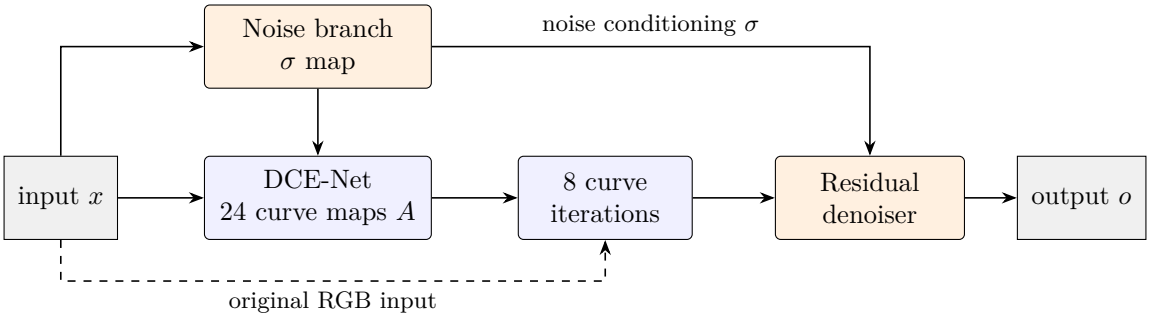


Figure 2: NTA-ZeroDCE. Orange blocks (noise branch, denoiser) are optional extensions; with them disabled the model is the Zero-DCE baseline. The dashed arrow shows that the curves are applied to the original input.

the curves applied (brightening amplifies noise by about that gain). The final output is $o = \mathrm{clamp}(z - 0.1 \tanh(\mathrm{Denoiser}(\cdot)), 0, 1)$.

- **Light variant.** An optional depthwise-separable DCE-Net and a `curve_scale` option estimate the curves at reduced resolution and up-sample them, which is used in Sec. 5.9.

2.3 Training losses

Training needs no reference images. The four baseline losses follow Zero-DCE:

- **spa** (weight 1): neighbouring 4×4-pooled contrast must be preserved between input and output.
- **exp** (weight 10): $\overline{(P_{16}(Y(o)) - E)^2}$ pulls the mean intensity of 16×16 patches toward a target E (the "exposure target").
- **col** (weight 5): penalises differences between the mean R, G, B channels (grey-world colour constancy).
- **tv** (weight 200): smoothness of the curve maps A.

Our extensions, each behind a feature flag in the configuration:

Flag	Loss	Idea	Weight
`noise_aware`	noise, invariance	Add synthetic signal-dependent noise ($\text{var} = ax + b^2$) so the true σ is known; the noisy output must match a no-gradient clean-input teacher	20, 10
`denoise`	fidelity	Denoised output stays close to the teacher's raw output	1
`edge`	edge	Output gradients follow gain-scaled input gradients, down-weighted where the gradient is noise	20
`chroma`	chroma	Output chromaticity $v/\sum v$ follows the locally averaged input chromaticity (prevents colour shifts)	20
`adaptive_exp`	adaptive_exp, highlight	Replaces *exp* by a per-region target that rises for bright and falls for noisy regions; penalises brightening bright pixels	10, 20
`temporal`	temporal	L1 between the output and the flow-warped previous output on reliable pixels	10

Targets, masks, σ and the gain used inside the losses are detached. The total loss is the weighted sum of enabled terms, minimised with Adam (learning rate 10^{-4}, weight decay 10^{-4}, gradient clipping 0.1).

2.4 Video consistency

For video, an OpenCV DIS optical-flow estimator is run in both directions; a pixel is "reliable" if the forward/backward flows agree, the photometric difference is small and the pixel stays in bounds. At inference the curves of the current frame are blended with the warped, smoothed curves of the previous frame, $\bar{A}_t = (1 - 0.7m)A_t + 0.7m\,\text{warp}(\bar{A}_{t-1})$ with m the reliability mask; a histogram-based scene-cut detector resets the state. A still image never uses flow or state. The temporal path is implemented and tested, but the shipped photo model was trained without it (see Sec. 5.1).

3 System implementation

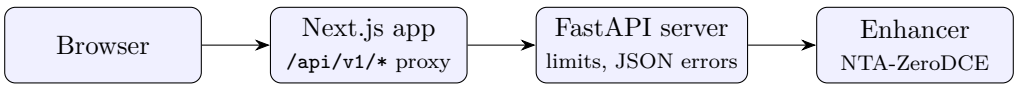


Figure 3: Deployment architecture. The web layer never runs the model; it proxies requests at run-time to the API (`LUMINA_API_URL`).

- **Python package** `nta_zerodce` (`core/`): modules for models, losses, data, flow, inference, metrics, training, API and a shared CLI. Dependencies are one-directional (e.g. models never import losses or data). Experiments are Python files with a strict schema: unknown or missing keys fail, flag dependencies are checked, and each run saves its resolved `config.yaml`.
- **Scripts:** `train`, `evaluate`, `enhance`, `ablate`, `make_synthetic_data`, `download_lol`, plus three added for checkpoint review: `benchmark_inference` (speed and pixel drift versus `curve_scale`), `prepare_photo_validation` (builds the held-out normal-light preservation set) and `check_photo_regressions` (the per-image promotion gate; Sec. 5.4).
- **API** (FastAPI): `GET /health`, `GET /options`, `POST /enhance` (PNG/JPEG upload with optional `curve_scale`, returns an 8-bit PNG). Uploads are limited in bytes, pixels and side length; admission is checked *before* multipart parsing, so a saturated server answers `503 server_busy` with `Retry-After` instead of queuing uploads in memory; errors are always JSON with fixed codes; requests carry an id and `Cache-Control: no-store`. Checkpoint, device and limits are set through environment variables (an `.env` file lists

every local checkpoint with one active line), and a Docker image is provided. We found and fixed a concrete reliability bug here: the installed Starlette version had dropped the `max_part_size`/`spool_max_size` multipart-parser interface the upload handler relied on, so every `POST /enhance` request raised an unhandled `TypeError`; pinning `starlette>=0.46.0` and `python-multipart>=0.0.18` restored it (Sec. 5.6).

- **Web app** (Next.js, TypeScript): image upload with client-side validation and down-scaling, a draggable before/after slider, and an Analysis panel computed in the browser.
- **Quality assurance:** 2,080 Python tests (numerical identities, gradients, curve and affine conventions, checkpoint round trips, a 30-step overfit test, an end-to-end train/enhance/evaluate cycle, API contract and error codes, plus new tests for the benchmark and validation scripts) and 70 web tests (Vitest), plus linting and a GitHub Actions workflow that runs the full suite, including the API/end-to-end tests, on Python 3.10 and 3.12. We re-ran both suites for this report: 2,080 Python tests passed (2 skipped, CUDA-only) and all 70 web tests passed. CI's `mypy` step runs only on the Python 3.12 job; in our local 3.13 environment `mypy -strict` reports 36 pre-existing errors, almost all requiring explicit `ndarray[...]` generics that newer numpy stub versions demand project-wide. Whether these reproduce in CI's own dependency resolution was not checked for this report; either way they are unrelated to the changes described here.

4 Experimental setup

Data. We use the public LOL dataset [3]. The 485 low-light training images are used *without* their references (the objective is zero-reference). The 15 held-out low/normal pairs (`eval15`) are used only for evaluation. A procedural synthetic dataset (analytic affine motion with known flow and exposure flicker) was used to train the temporal variant.

Metrics. PSNR and SSIM (11×11 Gaussian window, $\sigma = 1.5$) against the normal-light reference; angular error (degrees between RGB vectors, a colour-accuracy measure, lower is better); mean output brightness; fraction of clipped pixels ($Y \geq 0.99$); and a flat-region noise estimate (Immerkær's method [5]), lower is better but biased by texture. All numbers are mean ± standard deviation over the 15 images. Because the model is deterministic at inference and each setting was trained once (seed 17), the standard deviation reflects variation *between images*, not between training runs.

Settings common to all runs. Adam, learning rate 10^{-4}, batch size 8, 128×128 random crops, 32 channels, 8 curve iterations, loss weights as in Sec. 2.3, CPU training. The checkpoints in Table 1 and the epoch sweep (Sec. 5.2) all use `loss.exposure_target` = 0.6; Sec. 5.5 reports a later, separate 10-epoch run at 0.5. Evaluation ran on CPU at full resolution (400×600).

Baselines. Input (no processing); histogram equalisation of the luma channel; and gamma correction x^γ with $\gamma \in \{0.5, 0.4, 0.3\}$.

Run	Training data	Extensions on	Epochs
`lol_dark_only`	485 LOL low-light photos	noise, denoise, edge, chroma, adaptive exp.; *no* temporal	Config said 60; we stopped at 40 and kept a checkpoint every 5 epochs
`lol/final.pt` (shipped)	same	same	10 (identical weights to epoch 10 of the above)
`full` (checkpoint later removed)	96 synthetic procedural pairs	all, including temporal	40
`photo_target05_20261189`	485 LOL low-light photos (same split)	same as `lol_dark_only`	10, exposure target 0.5 (Sec. 5.5)

Table 1: The training runs referenced in this report. `full`'s checkpoint flattened real photos to grey and was removed from the repository after this report's figures were produced from its logs (`checkpoints/README.md`); only the logged curves in Figs. 4–5 remain as a historical record, and that run is not reproducible from the current repository state.

5 Results

5.1 Training dynamics

Fig. 4 shows the loss logs of the two runs for which we have complete logs. The total loss on LOL falls quickly during the first 10 epochs, then slowly (validation total 4.30 → 2.30 after 40 epochs). The synthetic/temporal run reaches a lower value (1.97), but the two are *not directly comparable*: the data differ and the second run also optimises the temporal term. The decomposition of the validation loss (Fig. 5) shows that the noise, chroma, edge and adaptive-exposure terms dominate the weighted loss, while the baseline spa and colour terms contribute almost nothing. This is a sign that the weights are unbalanced (they were set by hand and never tuned).

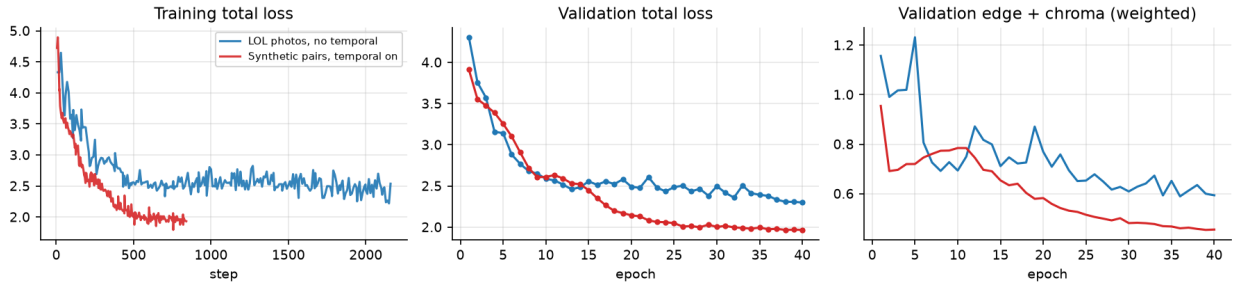


Figure 4: Left: training loss per logged step. Middle: validation loss per epoch. Right: the weighted edge and chroma terms of the validation loss.

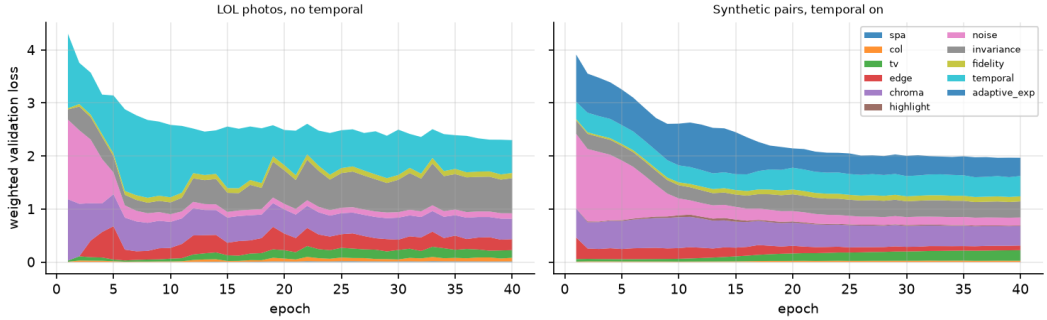


Figure 5: Stacked weighted validation-loss components for both runs.

5.2 Tuning the training length (checkpoint sweep)

The most important "setting" we tuned was how long to train. We evaluated each of the eight saved checkpoints of the `lol_dark_only` run on the 15 held-out pairs (Table 2, Fig. 6).

Setting	PSNR (dB) ↑	SSIM ↑	Ang. err. ↓	Brightness	Clipped	Noise est.
Input (no processing)	7.77	0.195	10.36	0.059	0.000	0.005
Histogram equalisation	14.24	0.419	9.98	0.512	0.008	0.053
Gamma $\gamma = 0.5$	12.33	0.563	9.17	0.216	0.000	0.014
Gamma $\gamma = 0.4$	14.59	0.622	9.28	0.285	0.000	0.017
Gamma $\gamma = 0.3$	**16.70**	**0.643**	9.45	0.379	0.000	0.021
Ours, epoch 5	10.93	0.504	**7.17**	0.175	0.000	0.010
Ours, epoch 10 (shipped)	10.68	0.506	7.77	0.183	0.000	0.008
Ours, epoch 15	13.54	0.561	7.76	0.280	0.000	0.010
Ours, epoch 20	15.86	0.593	7.49	0.444	0.000	0.020
Ours, epoch 25	14.90	0.588	7.45	0.491	0.000	0.020
Ours, epoch 30	15.68	0.581	7.75	0.425	0.000	0.016
Ours, epoch 35	16.19	0.593	7.42	0.481	0.000	0.021
Ours, epoch 40	16.58	0.591	7.29	0.469	0.000	0.021

Table 2: Held-out LOL results (mean over 15 image pairs; per-image standard deviations are in Fig. 6). The mean brightness of the ground-truth references is 0.451. Best value per column among all rows in bold except the angular error, where bold marks the best of our models.

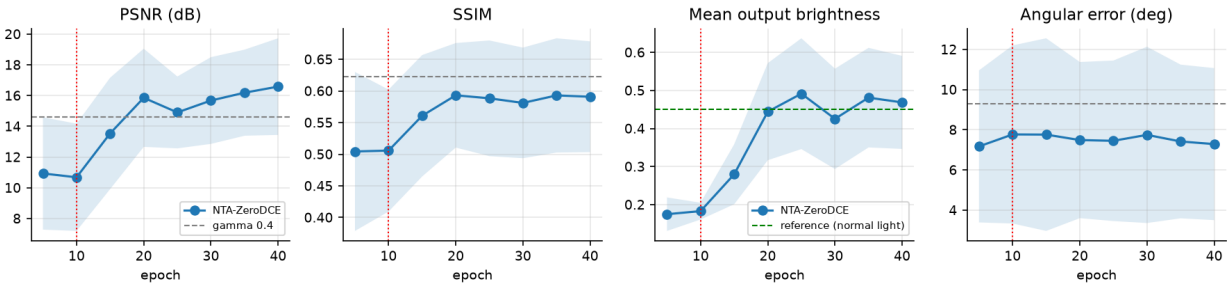


Figure 6: Metrics versus training epoch. Shaded band: ±1 standard deviation across the 15 images. The dashed grey line is gamma 0.4; in the brightness panel the green dashed line is the average brightness of the ground-truth references. The red dotted line marks the shipped epoch-10 model.

Observations.

- PSNR rises from 10.7 dB at epoch 10 to 16.6 dB at epoch 40, with a visible jump between epochs 10 and 20; SSIM plateaus around 0.59 from epoch 20.
- Mean brightness climbs from 0.18 to ≈ 0.47 (the reference average is 0.45). The long-trained model therefore does what the benchmark wants on these *very dark* images: it brightens a lot.
- Angular error (colour accuracy) is roughly constant, 7.2 to 7.8 degrees, and about 1.5 to 2 degrees better than gamma or histogram equalisation, whose colours shift with brightness.
- The noise estimate doubles as brightness rises, as expected: brightening amplifies noise. Histogram equalisation produces by far the noisiest images (0.053).
- Clipping stays at essentially zero on these dark images, so this test set cannot reveal the over-brightening problem discussed next.

5.3 Why we nevertheless ship the 10-epoch model

The LOL evaluation images are *extremely dark* (mean 0.06), so a stronger model scores better on them. Ordinary photographs, such as those users upload to the web app, are not that dark. In

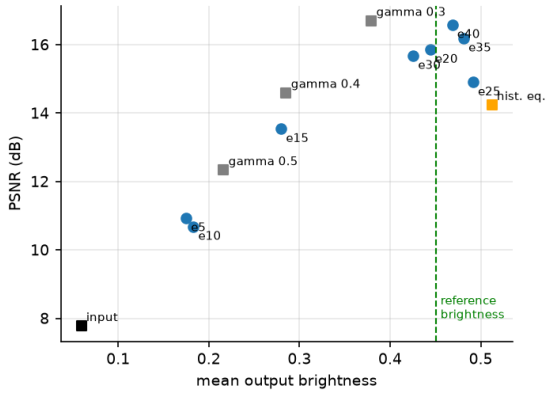


Figure 7: PSNR against mean output brightness. PSNR is largely governed by how close the overall brightness comes to the reference: it increases with brightness for gamma as well as for our checkpoints.

our manual tests (recorded in the project notes), the checkpoints from epoch 15 onward start to blow out moderately dark photos, and by epoch 25 they wash out almost completely (flat, grey, low-contrast images). Epochs 5 and 10 give natural, moderate enhancement but under-brighten the darkest scenes. This is why the benchmark score and the visual quality move in *opposite* directions, and why `lol/final.pt` is the epoch-10 checkpoint. When this report was first written we had no held-out measurement for this, only visual inspection; Sec. 5.4 describes the preservation gate we built since and the measured regression it catches between epoch 10 and 15. Fig. 8 shows the progression on LOL images, and the epoch-25 and epoch-40 columns already look bright and low-contrast even though they have the best PSNR.

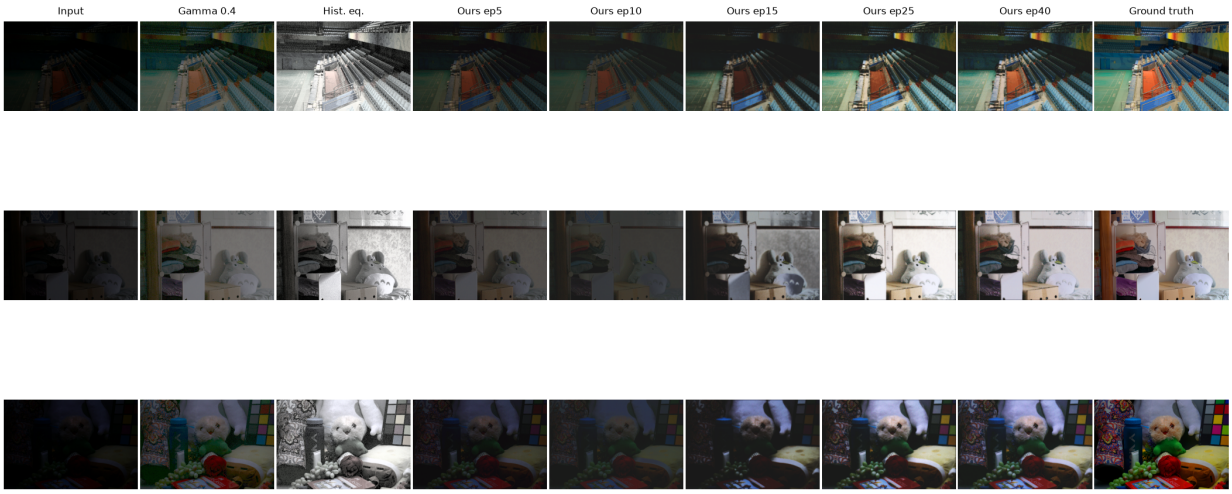


Figure 8: Three held-out LOL images processed by the baselines and by different checkpoints. The shipped model (epoch 10) is conservative; later epochs are brighter and score higher on PSNR.

5.4 A reusable preservation gate for checkpoint promotion

Rather than keep relying on eyeballing a handful of photos, we built a small pipeline so that a brightening regression can be measured before a checkpoint is promoted. `prepare_photo_validation.py` takes the 48 scenes that the trainer's fixed scene split (Sec. 4, seed 17, 10% held out) already excludes from training, and writes their *normal-light* LOL counterparts (never trained on, by construction) as a hashed, append-only manifest. `evaluate.py -unpaired` then reports, per image, mean brightness and contrast (and their input values) and the clipped-pixel fraction. `check_photo_regressions.py` compares a candidate checkpoint's per-image numbers against a

reference checkpoint's, flagging any image whose brightness rose, contrast fell or clipping grew by more than an operator-supplied limit; it never trains, ranks or auto-deploys anything, and it exits non-zero on a regression.

This set is a held-out split *within LOL*, not an independent collection of ordinary photographs, so it tests preservation on normal-light scenes from the same source rather than true out-of-distribution generalisation (see Limitations). As a demonstration, we ran it on the `lol_dark_only` epoch-10 and epoch-15 checkpoints (illustrative limits: +0.05 brightness, −0.02 contrast, +0.01 clipping). Mean brightness over the 48 scenes rose from 0.554 (epoch 10) to 0.577 (epoch 15, input mean 0.493), and four individual images failed the limits: two from excess clipping (scenes 126, 53) and two from excess brightening (598, 769) that a dataset-average comparison would not have shown on its own. A self-comparison of epoch 10 against itself passed, as a sanity check on the gate. This is consistent with, and now partly quantifies, the over-brightening judged by eye in the previous section.

5.5 Testing the exposure-target mitigation

The exposure loss `exp` (Sec. 2) pulls every patch's mean intensity toward one fixed target E, regardless of content; the obvious hypothesis is that a lower E should reduce the pressure toward over-brightening on already-reasonable patches. We tested this directly: a second 10-epoch, seed-17 run (`photo_target05_20261109`, otherwise identical to `lol_dark_only`'s recipe) with $E = 0.5$ instead of the shipped 0.6, evaluated with the same pipeline on the 48-scene preservation set and the 15 paired LOL images.

Measurement	Shipped (target 0.6)	Candidate (target 0.5)
Normal-photo SSIM vs. input (48 scenes)	0.9353	**0.9464**
Normal-photo mean brightness (input 0.4927)	0.5542	0.5555
Normal-photo contrast std (input 0.1803)	0.1612	0.1700
Normal-photo clipped fraction	0.0103	0.0110
Normal-photo colour angular error vs. input	**1.67°**	1.74°
Low-light PSNR vs. reference (15 pairs)	**10.68** dB	9.51 dB
Low-light SSIM vs. reference	**0.506**	0.415
Low-light colour angular error	**7.77°**	8.24°

Table 3: Shipped (target 0.6) versus candidate (target 0.5), both after 10 epochs, seed 17. Bold marks the better value per row.

The candidate passed the illustrative preservation limits and nudged normal-photo structural similarity up, but mean brightness on normal photos barely moved, colour error on both sets got slightly worse, and – more importantly – held-out low-light PSNR and SSIM both *dropped* relative to the shipped model, with visibly weaker shadow detail on review. Lowering the exposure target did not fix over-brightening on ordinary photos and cost accuracy on the dark images the model is nominally trained for, so we did not promote it: `lol/final.pt` (target 0.6) remains the shipped model, and target 0.5 is kept only as a documented, rejected experiment (`configs/photo_target05.py`). This is a single seed, single run; it rules out the simplest version of the fix, not the whole idea of adjusting the exposure target.

5.6 Fixing a real API reliability bug

While exercising the server end-to-end for this round of work we found that every `POST /enhance` request failed with an unhandled `TypeError`: the upload handler builds Starlette's `MultiPartParser` with a `max_part_size` constructor argument and then sets `parser.spool_max_size` to keep uploads in memory, but the Starlette version resolved by the project's open-ended `>=0.30` dependency bound no longer exposes either. This is independent of the model and of everything

else in this report, but it meant the deployed API could not serve a single enhancement request. We fixed it by raising the minimum to `starlette>=0.46.0` (which restores that constructor argument) and `python-multipart>=0.0.18`, and added regression coverage so a future dependency drift fails CI instead of failing silently in production.

5.7 Comparison with simple baselines

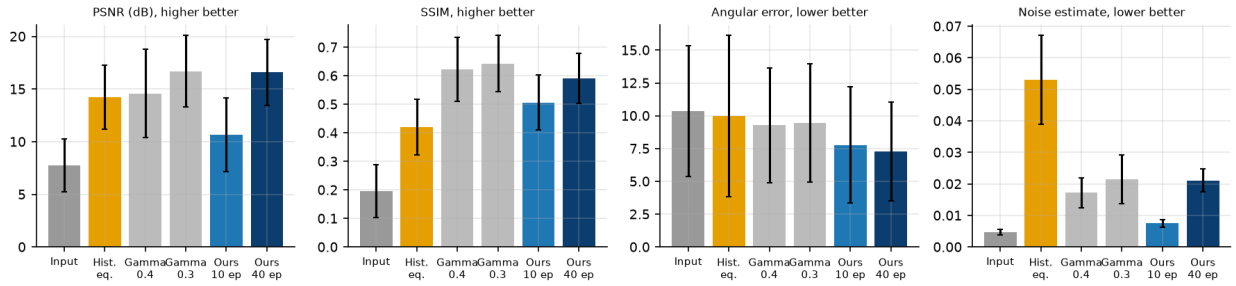


Figure 9: Methods compared on the 15 held-out pairs (error bars: standard deviation across images).

Honest reading of Table 2: with respect to PSNR and SSIM, a one-parameter gamma correction ($\gamma = 0.3$: 16.7 dB, 0.643) is as good as or better than our best checkpoint (16.6 dB, 0.591). Our model's measurable advantages are (i) a lower angular error (better colour preservation, 7.3 vs 9.5 degrees), (ii) no clipping or noise explosion compared with histogram equalisation, and (iii) spatial adaptivity: it does not apply one global curve. Because a zero-reference model is trained without the reference, PSNR is a secondary indicator; but we cannot claim that it beats a tuned global gamma on this benchmark.

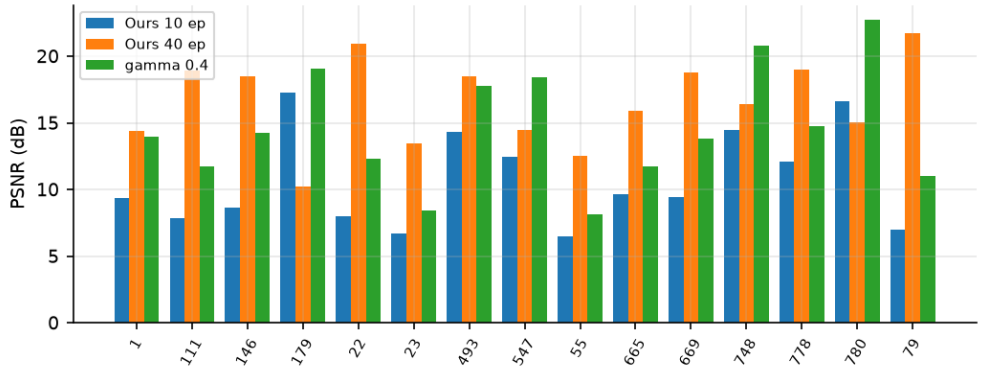


Figure 10: Per-image PSNR for the epoch-10 model, the epoch-40 model and gamma 0.4. The large spread (note the standard deviations of 3 to 4 dB in Table 2) shows that a 15-image test set is small and results should be read with caution.

5.8 What the model learns: curves, noise map and denoiser

Fig. 12 visualises the internals for one test image (LOL 778) for the epoch-10 and epoch-40 checkpoints. The mean curve magnitude $|A|$ grows from 0.10 to 0.27 between epoch 10 and 40, which is the mechanism behind the brightening drift: the network simply learns stronger curves. The noise branch predicts a mean σ of about 0.019 (10 ep) and 0.016 (40 ep); its map is lowest in the darkest, smooth regions and higher in brighter, textured regions, so it partly reacts to image structure and not only to noise. On this image the residual denoiser improves PSNR from 9.7 to 12.1 dB (10 ep) and from 14.9 to 19.0 dB (40 ep), compared with the curve output alone. This is a single-image observation, not a dataset-level ablation.

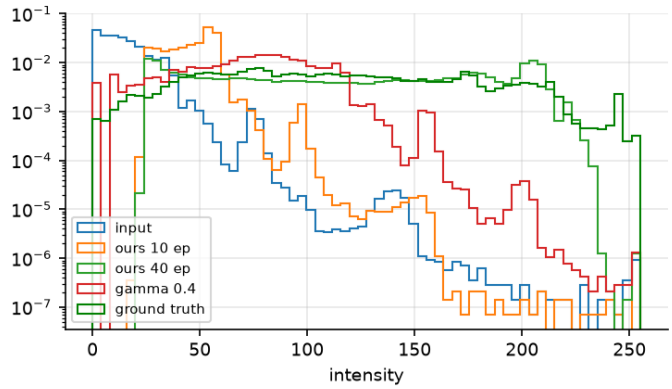


Figure 11: Intensity histograms (log scale) over all 15 test images. Inputs concentrate near zero and the shipped model shifts the mass only moderately (mostly below 100). The epoch-40 model's histogram is closer to the ground truth in the mid-tones, but it has no pixels below about 20 and almost none above about 215, i.e. reduced contrast with no deep blacks, which is consistent with the washed-out look.

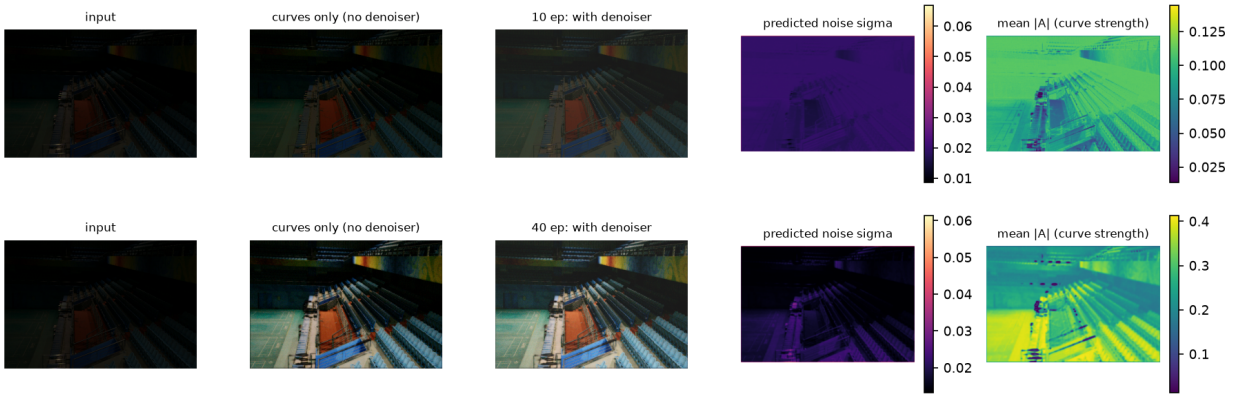


Figure 12: Inputs, curve-only output, final output, predicted noise map σ and mean curve strength for epoch 10 (top) and epoch 40 (bottom).

5.9 Inference-time setting: `curve_scale`

`curve_scale` estimates the curves at a reduced resolution and bilinearly up-samples them, which speeds up inference. We evaluated the shipped model with `curve_scale` $\in \{1, 0.5, 0.25, 0.125\}$ (Table 4, Fig. 13).

`curve_scale`	PSNR	SSIM	Ang. err.	CPU s/image
1.0	10.68	0.506	7.77	0.76
0.5	10.71	0.513	7.81	0.39
0.25	10.72	0.517	7.85	0.29
0.125	10.74	0.520	7.90	0.34

Table 4: Effect of `curve_scale` on LOL accuracy (shipped model, 400×600 images, CPU).

Quality is practically unchanged: PSNR and SSIM slightly *increase* (smoother curves), and the colour error increases by 0.1 degrees. Speed roughly doubles at 0.5 and nearly triples at 0.25; below that no further gain appeared. Timings in Table 4 were taken on a shared, unloaded-but-uncontrolled desktop CPU and fluctuate (the 0.125 point is slower than 0.25), so they show the trend rather than precise values.

We later added `benchmark_inference.py`, a dedicated script that records checkpoint hash, image paths, runtime versions, one warm-up and three timed runs per image, and the per-

pixel drift against `curve_scale=1` on the same checkpoint (not against a reference photo). Table 5 repeats the comparison at 1 and 4 CPU threads on six committed ordinary photos (`public/low-light-*.png`), which is a different, more controlled measurement than Table 4's LOL timings.

Threads	`curve_scale`	ms/image	Speedup	Max. pixel drift (0–255)
1	1	1544.2	1.00×	0
1	0.5	428.0	3.61×	75.95
1	0.25	224.5	6.88×	90.67
4	1	1250.2	1.00×	0
4	0.5	425.0	2.94×	75.95
4	0.25	165.4	7.56×	90.67

Table 5: Speed and drift versus `curve_scale` and thread count (same CPU as Table 4, measured 9 Nov. 2026). Drift is mean-absolute and max per-pixel-per-channel difference from the same checkpoint at scale 1, not error against a reference photo.

Average drift stayed below one intensity level even at 0.25 (not shown; see the raw JSON), but the *maximum* per-pixel drift (up to 91 of 255) shows that small average drift can hide locally large differences, consistent with the near-zero average PSNR/SSIM change in Table 4 not ruling out visible local changes. We keep `curve_scale=1` as the default for this reason and treat lower scales as an explicit, inspected speed trade-off rather than a free win.

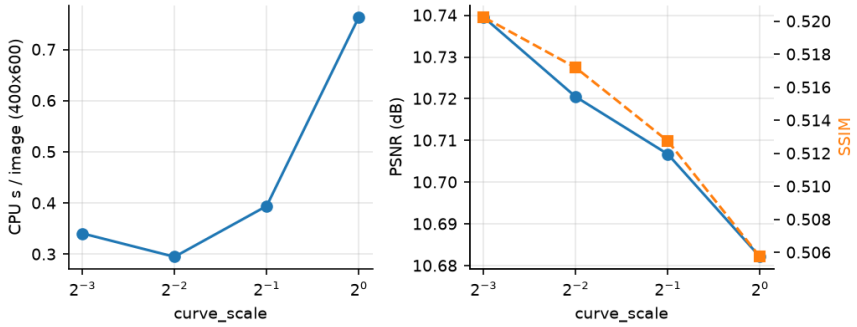


Figure 13: Speed and quality against `curve_scale`.

5.10 Other settings we tried

Table 6 lists the remaining decisions, including those taken from the project history and notes rather than from experiments re-run for this report. The right-hand column says what evidence exists.

Table 6: Settings tried, outcomes and evidence.

Setting	Outcome	Evidence
Training data: 96 synthetic procedural images, 40 epochs	Model flattened real photographs to grey; the checkpoint was removed	Project notes (`checkpoints/README.md`)
Training data: LOL low-light photos	Works; basis of all shipped weights	Sec. 5.2
Adding normal-light and darkened copies to the training set	Did not fix the over-brightening	Project notes (`docs/ARCHITECTURE.md`); not re-run
Training length 5 / 10 / 15 / … / 40 epochs	10 chosen for visual quality; 40 for benchmark score	Table 2

Continued on the next page

Setting	Outcome	Evidence
`exposure_target` 0.6	Used for *all* shipped checkpoints; retained after testing 0.5	Run configs; Sec. 5.5
`exposure_target` 0.5 (completed)	Retrained 10 epochs, seed 17: passed the preservation gate but *lowered* low-light PSNR/SSIM and lost shadow detail; **not promoted**	Table 3; Sec. 5.5
Per-step brightness/clipping diagnostics in training logs	Added so drift is visible during training; monitoring only, not a fix on its own	`training/loop_step.py`
Held-out normal-light preservation gate (48 LOL scenes, per-image limits)	Built and exercised on epoch 10 vs. 15; caught 4 per-image regressions a dataset average would hide	Sec. 5.4
Starlette version bound (`>=0.30`)	Silently broke the upload parser (every `POST /enhance` failed); pinned `>=0.46.0`	Sec. 5.6
Temporal term on synthetic pairs	Run trained; lower validation loss, not comparable (different data)	Fig. 4
`curve_scale` 1 to 0.125	No LOL-accuracy loss, 2–7× faster depending on threads; local pixel drift up to 91/255	Tables 4–5
13-row ablation matrix (3 seeds)	Defined in `configs/ablation_matrix.py`; **still not run**	No results exist

6 Discussion

Benchmark score versus perceived quality. The central lesson is that optimising or selecting by held-out PSNR on very dark images would have chosen a 40-epoch model that visually over-brightens. Because the training losses contain an exposure term with a fixed target (0.6), the longer the network trains, the more it pushes every image toward that mean. Stronger curves (Fig. 12) are the mechanism. Early stopping at epoch 10 is a workaround, not a fix.

The obvious fix did not work. We tested the most direct mitigation, lowering `exposure_target` to 0.5 (Sec. 5.5): it did not meaningfully reduce normal-photo brightening and it cost accuracy on the dark images the model targets, so the shipped recipe keeps 0.6. This is informative in the way a negative result is informative – the fixed-target exposure loss itself, not just its numeric value, is the more likely culprit, pointing toward the remaining untried ideas: (i) use the adaptive-exposure target more aggressively so already well-lit regions stay unchanged, instead of only replacing a global constant with a smaller global constant; (ii) build a validation set of genuinely independent ordinary photos (not LOL's held-out normal-light scenes, which share LOL's camera and processing) and stop on brightness/clipping instead of LOL PSNR; (iii) include a small fraction of well-exposed images with an identity loss during training. What *is* done, and reusable for testing any of these, is the preservation gate itself (Sec. 5.4): any future candidate can be checked against the shipped model on the same 48 scenes before being considered for promotion.

The baselines are strong. A tuned gamma beats our model on PSNR and SSIM on LOL. The model's value lies in colour fidelity, spatially varying enhancement, a noise-aware path and the fact that it needs no tuning per image, but a fair claim requires a no-reference perceptual metric and a user study, which we did not do.

Limitations.

- Each setting was trained once with one seed; no confidence intervals over training runs. The exposure-target comparison (Sec. 5.5) is a single seed too, so it rules out that one run, not the idea in general.
- The 15-image paired test set is small; one set (LOL) with very dark images only.
- The 48-scene preservation set (Sec. 5.4) is a held-out split *within* LOL, not an independent

collection of ordinary photographs – it shares LOL's camera, processing and scene types, so passing it is necessary but not sufficient evidence of general preservation. The illustrative regression limits used there were also not independently validated against human judgements of acceptability.
- The ablation matrix (value of each extension) was not run, so the individual contribution of noise branch, denoiser, edge, chroma and adaptive-exposure losses is *not* established. The single-image denoiser result is anecdotal.
- Loss weights and thresholds are untuned starting values; synthetic noise is a Gaussian approximation.
- Video/temporal consistency was implemented and unit-tested, but no video benchmark was run.
- Checkpoints use PyTorch pickle serialisation: load only trusted files. The API has no authentication.
- `mypy -strict` reports pre-existing, unrelated errors locally (Sec. 3); it is not yet a hard gate project-wide.

7 Conclusion

We delivered a complete low-light enhancement system: a small, noise-aware zero-reference model, a robust API, and a web app with built-in analysis. Our experiments on checkpoint selection show a clear tension between a benchmark metric and natural-looking results, and we chose the conservative 10-epoch model (exposure target 0.6) for deployment. We also went one step further than diagnosing the problem: we built a reusable preservation gate, used it to test the most obvious mitigation, and found that mitigation does not work – a concrete, if negative, result that narrows where the real fix must come from. Separately, fixing the upload-parser bug (Sec. 5.6) was a reminder that an open-ended dependency bound can silently break a deployed feature with no code change on our side. Honest next steps are to run the ablation matrix, collect an independent set of ordinary photographs for the preservation gate, try the adaptive-exposure and identity-loss mitigations in Sec. 6, evaluate on video, and compare against other zero-reference methods.

Reproducing the numbers

Sec. 4's checkpoint-sweep and baseline numbers were produced by running the committed checkpoints in `core/checkpoints/lol_dark_only/` on the 15 LOL evaluation pairs in `core/data/LOL/eval15/` (CPU, full resolution); the `curve_scale` drift table (Table 5) by `scripts/benchmark_inference.py` on the six `public/low-light-*.png` images; and the exposure-target and preservation-gate results (Secs. 5.4–5.5) by `scripts/prepare_photo_validation.py`, `scripts/evaluate.py -unpaired` and `scripts/check_photo_regressions.py` against `core/checkpoints/lo` and `core/checkpoints/photo_target05_20261109/final.pt`; raw files are under `core/docs/benchmarks/`. The `full` run's training-curve figures (Figs. 4–5) are a historical record from before its checkpoint was removed (Table 1) and are not independently reproducible from the current repository.

References

[1] C. Guo, C. Li, J. Guo, C. C. Loy, J. Hou, S. Kwong, R. Cong. Zero-Reference Deep Curve Estimation for Low-Light Image Enhancement. *CVPR*, 2020.

[2] C. Li, C. Guo, C. C. Loy. Learning to Enhance Low-Light Image via Zero-Reference Deep Curve Estimation. *IEEE TPAMI*, 2021.

[3] C. Wei, W. Wang, W. Yang, J. Liu. Deep Retinex Decomposition for Low-Light Enhancement. *BMVC*, 2018.

[4] T. Kroeger, R. Timofte, D. Dai, L. Van Gool. Fast Optical Flow using Dense Inverse Search. *ECCV*, 2016.

[5] J. Immerkær. Fast Noise Variance Estimation. *Computer Vision and Image Understanding*, 1996.

[6] Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli. Image Quality Assessment: From Error Visibility to Structural Similarity. *IEEE TIP*, 2004.

[7] W.-S. Lai et al. Learning Blind Video Temporal Consistency. *ECCV*, 2018.

A Hyperparameters of the shipped configuration

Parameter	Value
Channels / curve iterations	32 / 8
Noise / denoiser channels	16 / 16
$\sigma_{\max}$, residual scale	0.2, 0.1
Gain window, gain range	5, [1, 8]
Loss weights (spa, exp, col, tv)	1, 10, 5, 200
Loss weights (edge, chroma, highlight, noise, invariance, fidelity, temporal)	20, 20, 20, 20, 10, 1, 10
Noise synthesis	$a \in [0, 0.01]$, $b \in [0, 0.02]$
Optimiser	Adam, $\beta = (0.9, 0.999)$, weight decay 10^{-4}
Learning rate, batch, crop	10^{-4}, 8, 128
Gradient clipping	0.1
Seed	17
Exposure target (shipped `lol`/`photo` recipes)	0.6
Exposure target (`baseline`/`full` research configs)	0.5
Exposure target (tested, rejected alternative)	0.5, `configs/photo_target05.py` (Sec. 5.5)
Starlette / python-multipart minimums	`>=0.46.0` / `>=0.0.18` (Sec. 5.6)